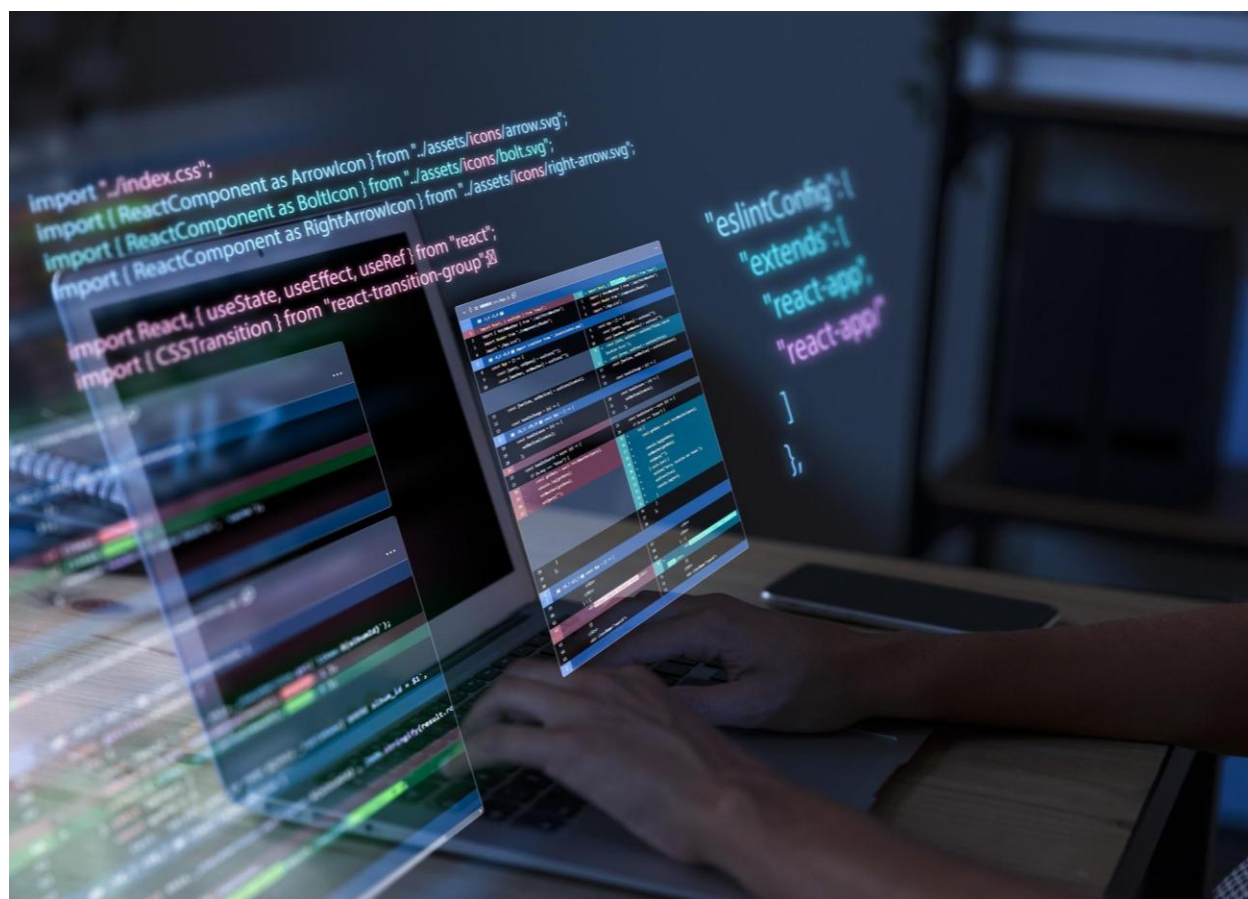


Ejercicios

POO Avanzada(I)



Autor: Marcela Martín

Fecha: febrero 2026

1. Crea un programa que modele un sistema básico de animales usando herencia.

a) **Crea una clase llamada Animal:**

- Tiene los atributos:
 - nombre (tipo String)
 - edad (tipo int)
- Tiene un método:
 - hacerSonido(): imprime "El animal hace un sonido genérico."

b) **Crea dos clases que hereden de Animal:**

- Perro:
 - Añade un atributo específico de los perros : raza (tipo String)
 - Crea el constructor con todos los atributos de Perro
 - Sobrescribe el método hacerSonido() para imprimir "El perro ladra: ¡Guau guau!"
 - Añade los métodos toString, getters/setters para todos los atributos
- Gato:
 - Añade un atributo específico de los gatos : peso (tipo double)
 - Crea el constructor con todos los atributos de Gato
 - Sobrescribe el método hacerSonido() para imprimir "El gato maúlla: ¡Miau miau!"
 - Añade los métodos toString, getters/setters para todos los atributos

c) **Crea una clase principal llamada AppAnimal:**

- Crea un objeto de cada clase (Animal, Perro, y Gato).
- Llama al método hacerSonido() de cada objeto.

2. Crea un programa que modele un sistema básico de vehículos usando herencia.

a) **Crea una clase llamada Vehículo:**

- Tiene los atributos:
 - marca (tipo String)
 - modelo (tipo String)

- Tiene los métodos:
 - `acelerar()`: imprime "El vehículo está acelerando."
 - `arrancar()`: imprime "El vehículo está arrancando"
 - b) **Crea dos clases que hereden de Vehículo:**
 - Coche:
 - Atributo específico: `numPuertas` (tipo `int`).
 - Constructor que incluya todos los atributos.
 - Sobrescribe el método `acelerar()` para imprimir "El coche acelera rápidamente."
 - Añade `toString()`, `getters` y `setters`.
 - Moto:
 - Atributo específico: `cilindrada` (tipo `int`).
 - Constructor con todos los atributos.
 - Sobrescribe el método `acelerar()` para imprimir "La moto acelera ágilmente."
 - Añade `toString()`, `getters` y `setters`.
 - c) **Crea una clase principal llamada AppVehiculo:**
 - Crea objetos de cada clase (`Vehiculo`, `Coche`, y `Moto`).
 - Llama a los diferentes métodos disponibles para cada objeto.
3. Crea un programa que implemente el modelo de una **Universidad** tenga varios **Estudiantes**, utilizando una relación de **agregación**.
- a) La clase `Estudiante` debe tener atributos como nombre, edad y matrícula.
 - b) La clase `Universidad` debe contener un array de estudiantes.
 - c) Agrega métodos para registrar estudiantes y mostrarlos.
4. Implementa un sistema en el que una **Biblioteca** contenga varios **Libros**, demostrando una relación de **agregación**.
- a) La clase `Libro` debe tener como atributos título, autor y año de publicación.
 - b) La clase `Biblioteca` debe contener un array de libros.
 - c) Agrega métodos para agregar libros y mostrarlos

5. Desarrolla un programa que implemente el modelo de un **Coche** compuesto por varias **Ruedas**, demostrando la relación de **composición** (las ruedas no existen fuera del coche).
 - a) La clase Rueda debe tener atributos como marca y tamaño.
 - b) La clase Coche debe contener un array de ruedas instanciadas en su constructor.

6. Desarrolla un programa que implemente el modelo en el que un **Ordenador** está compuesta por varios **Componentes** esenciales, con una relación de **composición** (los componentes no pueden existir fuera del ordenador).
 - a) La clase Componente debe tener como atributos: nombre, marca y capacidad.
 - b) La clase Ordenador debe contener un array de componentes creados en su constructor.